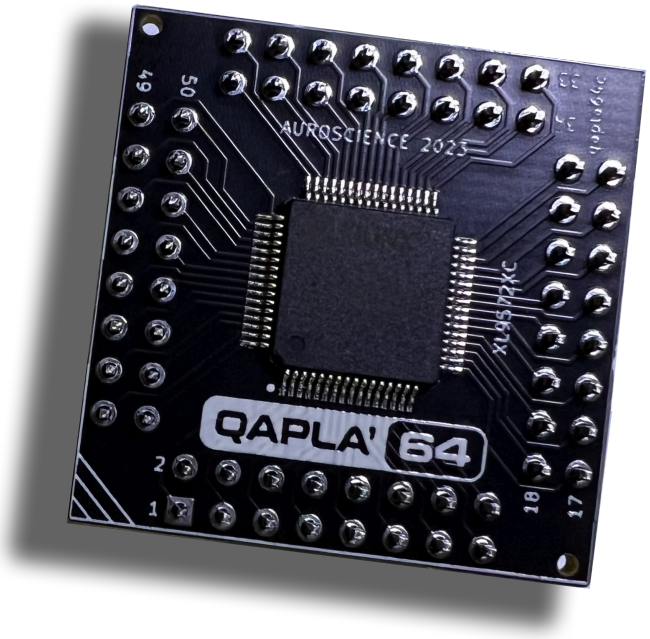
EVO 64
Auroscience Production



We Bid you QAPLA' (That's Klingon for "Success")

For anyone that isn't familiar with the Commodore 64's Programmable Logic Array (PLA), it's typically described as the "traffic cop" for the entire system. Its job is to tell chips when it's their time to wake up, so they can listen and communicate on the system's data and address buses. The PLA enables a bunch of cool tricks which have helped to make the C64 such a clever "success". Examples include bank switching that allows the CPU and the VIC-II video chips to access the system's memory independently, within a single, shared clock cycle.The PLA also enables the banking of the system's ROM images with main memory, as well enabling various cartridge types to interface system resources.

More information on the C64 PLA Chip
https://www.c64-wiki.com/wiki/PLA_(C64_chip)

Reworking the Classic

Over the years, there have been *many* attempts to replace the C64's PLA. This is mostly due to the MOS 906411-01 PLA chip, which is found in most of the early "Breadbin" systems (e.g. assembly 250407, 250425 and 250466), being among the most common chips to fail. A faulty PLA will typically trigger the dreaded "black screen of death", which has caused many users to throw their Breadbins into the proverbial dustbin.

The challenge with making a new PLA is to get all of the low level gate-logic to be 100% accurate, while ensuring that all of the output timings from the chip are perfect. There are some excellent resources and references available that can help with the core logic descriptions, but the timing tolerances can be pretty unforgiving, as just a few milliseconds too little or too much of a delay can cause a system to hang or freeze, or fail to boot entirely.

There have a number of great engineering efforts that have helped to demystify the inner workings of the PLA, from the early articles published in The Transactor magazine (March 1986, vol6, issue #5), to some great PLA white papers by Thomas 'skoe' Giesel and Jens Shoenfled, through to the many efforts to produce highly effective PLA clones, such as Francois Leveille's (aka Eslapion) PLAnktop and Passi Lassila's (aka 1c3d1v3r) NeatPLA, among others.

We can thank folks like them for performing a ton of testing and stepping through the painstaking process of sampling and monitoring the C64's PLA with their logic analyzers, under a litany of different conditions and system states, all while taking detailed notes of the precise timings, etc.

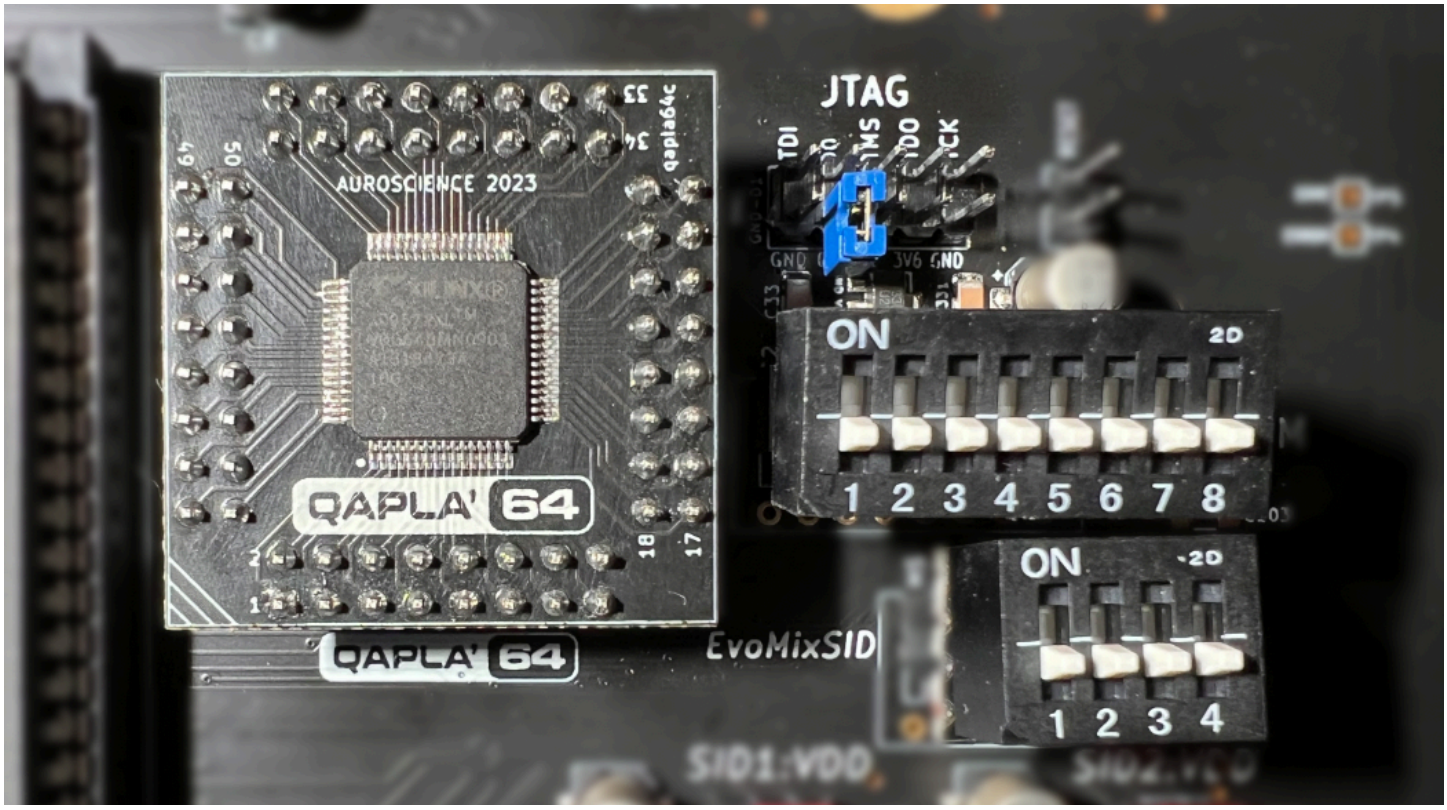
Going "BIG" or Going Home!

When we were researching and testing various PLA alternatives for the EVO design, we ultimately chose to go with a CPLD-based approach. This decision was based on the flexible-programmability and configurable performance that can be achieved with CPLDs. It also makes it possible to seamlessly integrate all of the additional logic functions that are necessary to support flexible addressing for multi-SID support, and a highly configurable set of ROM images and cartridge bank selections, without having to add a ton of extra discrete logic chips onto the mainboard itself.

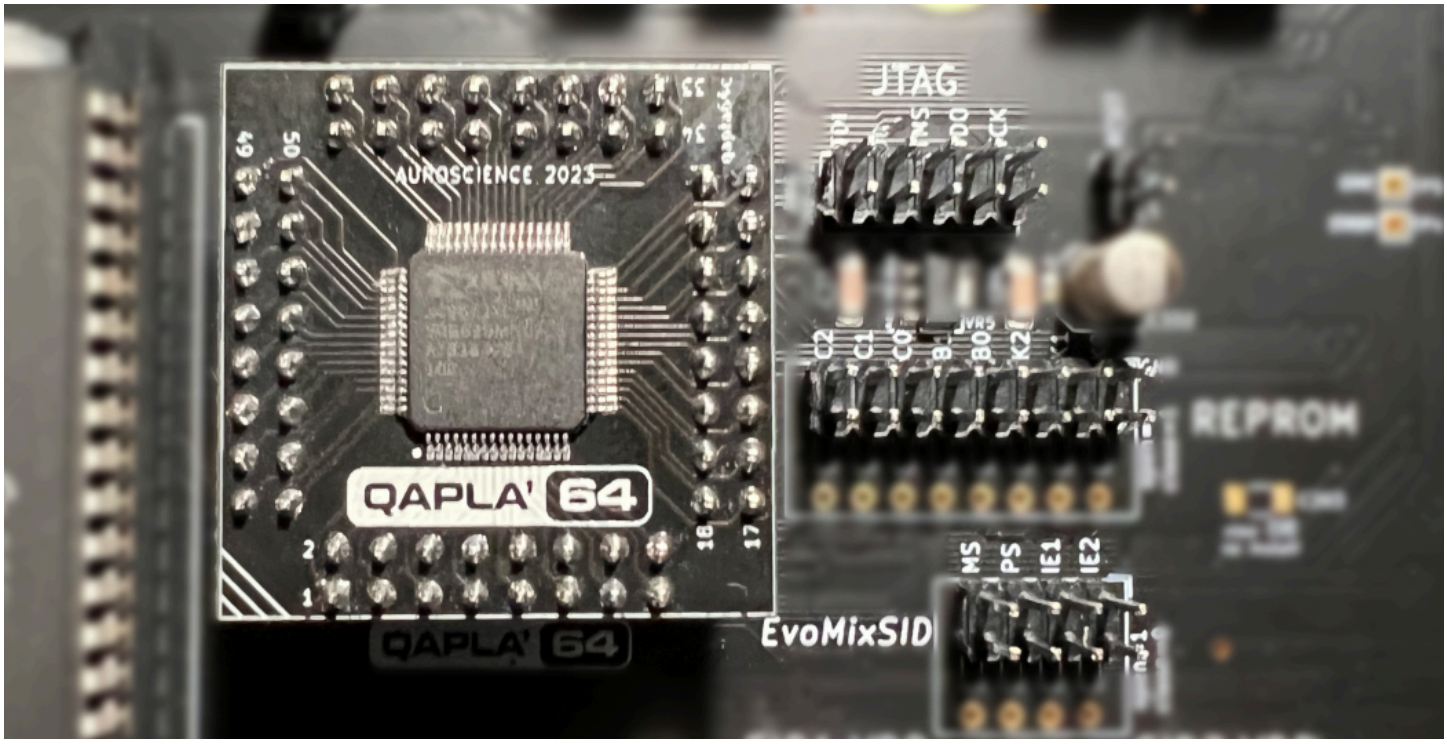
It's important to acknowledge the great work done by Henning Liebenau, on his **MixSID and RePROM64 designs**, which provided some early insights and inspiration for both of these solutions. However, the actual technical implementations that ultimately made their way into EVO's QAPLA' design are fundamentally unique and distinct, which have helped to make the QAPLA' a *one-stop-shop* for all of EVO's chip and system control logic.

The EVO team have rigorously tested QAPLA' for performance, compatibility and stability, across a wide range of timing-sensitive games and demoscene productions. So, at this stage we're quite confident that it's on par with the best PLA solutions available for the C64. When you combine this robustness with the added multi-SID and multi-ROM capabilities, we're happy to call QAPLA' *a success*!

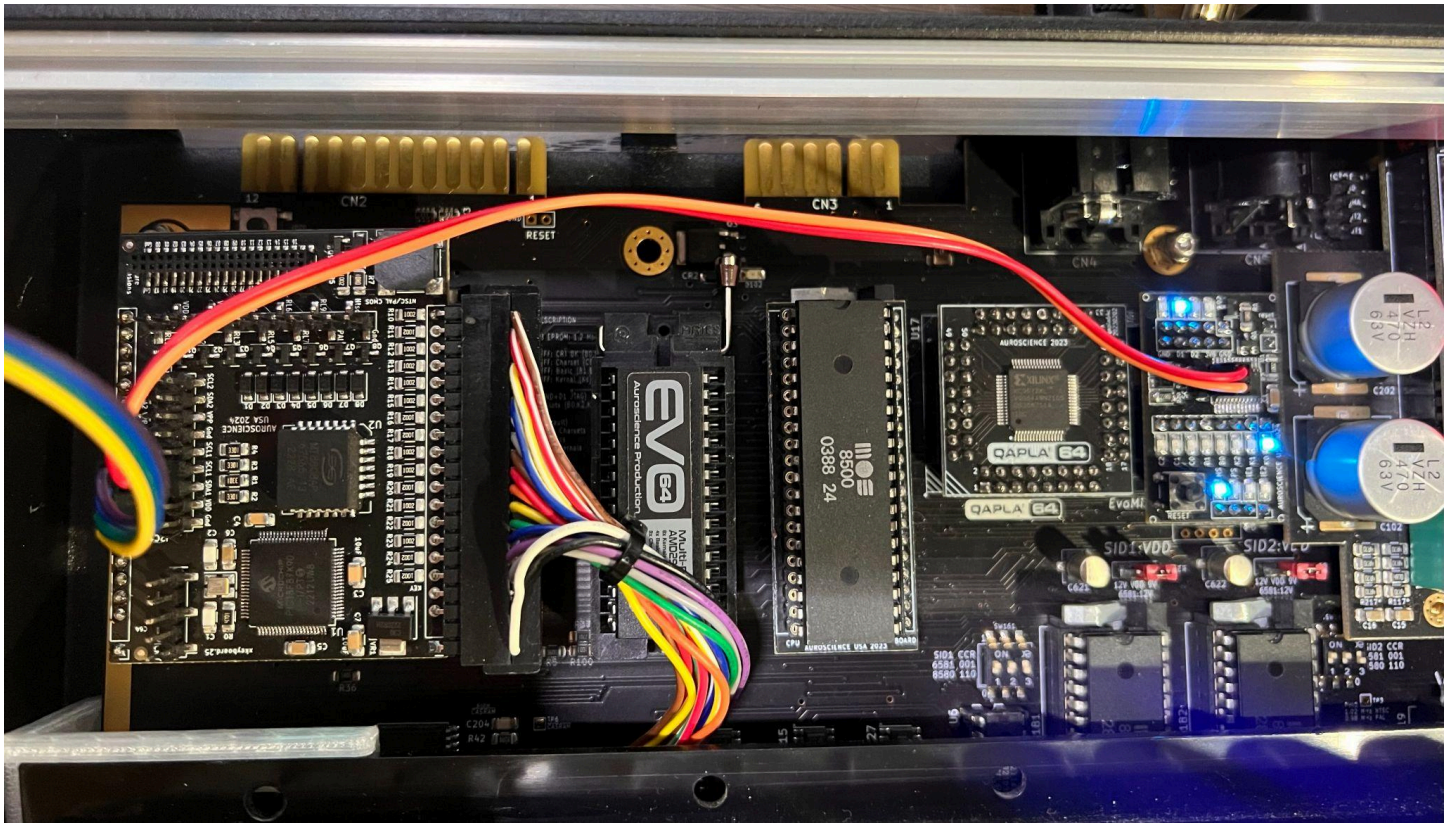
Using a set of dip-switch arrays with the QAPLA will make it easy to set logic positions using only your fingers



You can also choose to use standard 2.54mm pitch header pins, which will allow you to use jumper caps to set logic positions. This approach makes it possible to retrofit an automated ROM switching tool, such as the [EVO Multi-Switch-Module (MSM)](). Needless to say, building and configuring a KeyMan64 for use with EVO is beyond the scope of this document.



Naturally, we think the *best* option for managing all of your system settings is with an EVO Multi-Switch Module (MSM). The MSM comes with a ROM-SID module that you place on top of your configuration header pins. The ROM-SID module is connected to the primary MSM module, which intercepts the keystrokes from your keyboard. When you press the command-sequence hotkey and enter a command, your settings will change dynamically, *without having to touch your jumpers*.



Informative Links

EVO's optional **Multi-Switch-Module** for config state mgmt.

EVO's integrated **Multi-SID** implementation.

EVO's integrated **Multi-ROM** implementation.